IntentGate: Action Gating Against Prompt Injection and Tool Poisoning in Tool-Using LLM Agents

Anonymous Author(s)

***Abstract*—Tool-using agents built on large language models (LLMs) can be manipulated through instructions embedded in tool output or tool descriptions. As tools move into email, calendars, databases, and programming languages, this risk will continue to grow. Existing defenses modify the model itself, add a judge call after each agent action, or require a user to hand-author a policy for each tool. These methods do not scale well and do not provide complete protection. IntentGate is a lightweight middleware gate that extracts an intent contract directly from the user's trusted request before any malicious content is added. Each tool call is evaluated by its similarity to the extracted contract and a simple rule-based veto. Based on this evaluation, the call is allowed, blocked, or escalated to human review. This is done without requiring a direct LLM call for every action taken by the agent. The location of IntentGate provides a single point at which all tool calls occur, so the required policy comes from the user rather than from any content provided by an attacker. Evaluation was performed on two attack types: indirect injection on 2,108 InjecAgent cases and tool poisoning on 1,348 MCPTox cases. Under static attacks, attack success on InjecAgent falls from 19.0% to 0.7%, and attack-influenced MCPTox tool calls fall from 20.7% to 10.7%. Both benchmarks show large reductions. The gate adds 16–20 ms p95 CPU latency and needs no hand-authored per-tool contracts, compared with 145 for the ToolGate baseline. Additional testing on a second backbone (qwen3.5:4b) produced a comparable ASR reduction on InjecAgent (27.1% to 2.1%). The evaluation of benign utility, prompt baselines, and adaptive probes remains preliminary.**

***Index Terms*—action gate, intent contract, LLM agents, MCP, middleware, prompt injection defense, tool poisoning**

I. Introduction

LLM agents that use tools have access to the user's environment, including email, calendars, databases, and code, through a continuous cycle similar to ReAct [1]. At the same time, this access is also the attack surface. An attacker could send instructions via an email attachment, calendar invitation, database entry, or tool output, which are examples of indirect prompt injection [2], [3]. A malicious tool description in the Model Context Protocol (MCP) could direct the agent through tool poisoning, described on actual MCP servers [4], [5]. In either case the agent acts on the attacker's instruction without user authorization, which is the excessive-agency risk in the OWASP Top 10 for LLM applications [6]. Current defenses modify the model, make a judge call on the input, or require a hand-authored tool policy before execution, and a few derive the policy directly from the task [8]. IntentGate is a middleware gate that evaluates each tool call against an intent contract extracted from the trusted user request. It is evaluated on InjecAgent for indirect injection and MCPTox for tool poisoning.

Some prior defense approaches differ in how and where control is enforced. Model-based defenses modify either the model itself or its training data. However, CaMeL modifies the model by placing it within a planning-execution architecture that includes capability tracking [9]. StruQ also alters the model through fine tuning it to differentiate the task-specific instruction from the relevant input data [10]. Similarly, SecAlign uses preference optimization to achieve this same goal [11]. Models can be trained using an instruction hierarchy to force them to prioritize their most trusted instructions [12]. The above-mentioned defense strategies all require model cooperation and have not been directly compared against each other for use against InjecAgent or MCPTox.

The open question is whether a gate that scores each tool call against a contract from a single trusted request can reduce attacks on both benchmark classes. On InjecAgent, attack success fell from 19.0% to 0.7%, and on MCPTox attack-influenced tool calls fell from 20.7% to 10.7%. The gate adds 16–20 ms p95 CPU latency and needs no hand-authored per-tool contracts.

II. Background and Related Work

A. Threat model

This threat model is a tool-using LLM agent in a ReAct-style loop [1]. Based upon a trusted user request, the agent proposes potential tool calls, then runs those calls, receives output, and continues running calls until there is a result provided to the user. The attacker may introduce malicious content either through tool descriptions (tool poisoning) or through retrieved or returned content (indirect injection). Additionally, tool-selection attacks allow the agent to select a registered attacker tool even when the user's task is benign [15]. The attacker's goal is to make the agent execute an unauthorized tool call (data exfiltration, message sending, state changes, or credential exposure). The defender's goal is to block or escalate unauthorized tool calls without disrupting legitimate task completion. Everything following the initial request (tool definitions returned by a registry, tool outputs, injected document text) is potentially attacker-controlled. Attack content includes manually written text as well as optimized sequences [16], and earlier studies describe related failure modes in emulated agent environments [17].

B. Defenses

Model-level defenses track information flow at the model or runtime level. CaMeL [9] uses a planner-executor approach

with capability-based taint tracking, so data from attacker-controlled sources cannot reach tool arguments. StruQ [10] fine-tunes the model to separate instructions from data with structured queries, and SecAlign [11] applies preference optimization to the same end. Instruction-hierarchy training [12] teaches the model to prioritize privileged instructions. All of these require model cooperation, and none has a head-to-head adversarial evaluation on InjecAgent or MCPTox. Data-level defenses mark or transform untrusted text. Spotlighting [13] transforms retrieved content into clearly delimited forms. Guardrail toolkits such as NeMo Guardrails [14] enforce programmable rails around the model, but the model still makes the final call. Detection methods add training or inference cost: BIPIA fine-tunes a detector alongside its benchmark [18], and MELON re-executes with masked tools and compares outputs [19].

Judge-based defenses query a second language model to review each proposed call [20]. They are flexible, but they add cost and latency with a language-model call on every action. They also create a new attack surface, because the judge can be prompt-injected, and produce implicit policies that are hard to audit. Contract-based defenses come closest to where this work enforces control. ToolGate [21] enforces preconditions and postconditions for each tool at the tool-call boundary through hand-authored *per-tool contracts*. In this reimplementation the policy source is the weakness. Coverage is incomplete (65/801 MCPTox tools, 8.1% distinct and 37.2% availability-weighted), and attacker-registered tools have no contract. Progent version 3 [8] derives its initial symbolic policy from the user's task and updates it during execution. A solver-based check (SMT) permits narrowing automatically and requires approval for expansion. Its call check is symbolic, while IntentGate freezes its contract and uses embedding scores plus vetoes. ClawGuard [22] applies constraint specifications for tasks based on users' defined objectives. ClawGuard confirms all constraints via user interaction. ClawGuard has been applied to InjecAgent and MCPTox, so this work is not the first to derive policy from user input. Prompt Flow Integrity [23] includes agent isolation and data tracking to prevent privilege escalations. TrustAgent [24] utilizes a manually authored constitution, so its evaluation used emulation, not live tool execution. A recent design-pattern survey [25] concluded that secure agents prohibit unverified input from controlling sensitive operations.

AgentDojo [26], InjecAgent [27], and Agent Security Bench [28] provided the framework for the injection-based test cases. Wang et al. [4] developed MCPTox for tool poisoning. There is no mechanism for protecting against poisoned registrations (a weakness of the MCP) [5], [29]. Zhan et al. [30] demonstrated that adaptive attackers can defeat statically designed defensive systems. As such, the primary focus is static attacks, and preliminary results for an adaptive attacker probing exercise were achieved offline.

C. Research gap

ToolGate requires a hand-authored contract for each tool [21]. Progent creates a symbolic policy and modifies it in real time [8]. ClawGuard generates constraint-based policies (task-scoped) based on the user's goal, then asks the user to verify [22]. Task Shield implements a language-model judge on every action [20]. IntentGate parses the trusted input request into a single, immutable contract and enforces that contract using a CPU-embedded scoring system and a deterministic veto mechanism. As such, IntentGate does not require a new model to be developed or retrained, nor does it implement a new LLM for each action or tool. The authors' measured contribution was comparing the results of their implementation of IntentGate against an implementation of ToolGate on two benchmarks: an injection benchmark and a small subset of a tool poisoning benchmark. Additionally, they measured the setup costs directly. Specifically, they created 145 total contracts as part of the ToolGate baseline and zero as part of their IntentGate implementation.

III. Method

A. System overview and ordering invariant

IntentGate sits between the agent and the executor. Figure 1 shows the pipeline. The ordering invariant is strictly enforced. The intent contract is parsed from the trusted user request *before* tool definitions or any attacker content (injected observations, poisoned tool descriptions) are assembled. Tool outputs are never passed to the parser. Every proposed tool call, user-initiated and attacker-influenced alike, passes through `GateMiddleware.execute()` with no bypass path.

B. Intent parser and contract schema

A trusted intent parser (an LLM with temperature 0, JSON mode, one repair retry) reads the user request once per session and outputs an intent contract, the request-derived policy used by the gate.

$$C = (A, K, H), \tag{1}$$

In this contract, A is the set of allowed (entity, operation, resource) elements, K is a set of numeric and path constraints, and H is a set of high-risk flags. The implementation represents A as the contract's goals, expected tool categories, and permitted data access, K as argument constraints, and H as side-effect limitations. If a call has contact with an element outside of A, violates a constraint within K, or contains a flag within H, then it will fail the rule check and trigger the veto. The schema is versioned and locked for the evaluation. The contract is embedded only once during each session and then reused. Only the proposed call is embedded at decision time. Therefore, the agent's prompt remains cacheable and the gate is removed from the prompt.

TABLE I
RELATED-WORK COMPARISON: POLICY SOURCE, ENFORCEMENT, PER-ACTION LLM USE, BENCHMARKS, AND SETUP COST.

System	Policy source	Enforcement	LLM/action	InjecAgent/ MCPTox	Setup cost
CaMeL	Model-level	Capability taint tracking	No	No	Model change
StruQ	Model-level	Fine-tuned model	No	No	Model change
Task Shield	Task-scoped (LLM-derived)	LLM-as-judge	Yes	No	Per-task LLM calls
ClawGuard	Task-scoped (from user objective)	Deterministic rules	No	Yes	Once per request
Progent	Task policy (LLM-derived)	Symbolic checks, SMT updates	No (updates)	No	Automatic generation, approval
ToolGate	Manual pre/post contracts	Deterministic symbolic	No	No	145 contracts (InjecAgent)
Ours	**Auto-derived from user request**	**Embedding + rule veto**	No	Yes	**0 contracts**

C. Scorer

Each proposed tool call c is evaluated relative to the contract. The semantic component converts the cosine similarity between the contract embedding e_C and the call embedding e_c into $[0, 1]$:

$$S_{\text{sem}} = \frac{1 + \cos(e_C, e_c)}{2} = \frac{1}{2}\left(1 + \frac{e_C \cdot e_c}{\|e_C\| \, \|e_c\|}\right), \quad (2)$$

The rule component is also deterministic: $S_{\text{rule}} \in \{0, 1\}$. It is equal to 1 if the call satisfies all constraints in K and 0 otherwise. The overall score is

$$S = \alpha \, S_{\text{sem}} + (1 - \alpha) \, S_{\text{rule}}, \quad (3)$$

A veto results in setting $S = 0$ prior to thresholding. The default value is $\alpha = 0.7$. Both $\tau = 0.75$ and $\delta = 0.1$ are fixed values based on a 50-call pilot before any benchmark was performed. This study reports the operating point and does not assert that either τ or δ is optimal or that either value would be transferred across splits.

D. Gate middleware and escalate band

The middleware makes a decision for each call based on Equation 4:

$$\text{decision}(c) = \begin{cases} \text{BLOCK}, & S_{\text{rule}} = 0 \text{ or } S < \tau - \delta, \\ \text{ESCALATE}, & S_{\text{rule}} = 1 \text{ and } \tau - \delta \le S < \tau, \\ \text{ALLOW}, & \text{otherwise}, \end{cases} \quad (4)$$

The block case is fail-closed. The escalate case sends the call to human review and counts as a block in benchmark mode but is displayed to users in demo mode. Algorithm 1 shows the full procedure. Of the 583 escalated calls, 60.7% targeted attacker tools, 14.8% targeted user tools, and 24.5% targeted other tools. No additional LLM calls are made per action by the gate. As such, measured p95 per-call latency is 16–20 ms on CPU.

IV. TOOLGATE BASELINE (B2)

The main baseline has been a minimal reimplementation of ToolGate [21]: contracts with preconditions P and post-conditions Q on a symbolic state space, together with the ability to save snapshots and roll-back after a failed post-condition. Only the evaluated tool set has been covered by contracts. Of the user and attacker tools of InjecAgent (79/79), MCPTox registered tools (65/801) had 8.1% distinct and 37.2% availability-weighted. No silent allowance exists if no contracts exist: they have been accounted for as `no_contract` and logged. Instead of executing a ToolBench run, fidelity has been reviewed using contract reviews and recorded-call replays.

Hand-authored contracts were required for ToolGate: 145 on InjecAgent, and none were needed for IntentGate, since the contract was automatically generated based upon the request. Expanding contracts towards 84.9% weighted coverage on

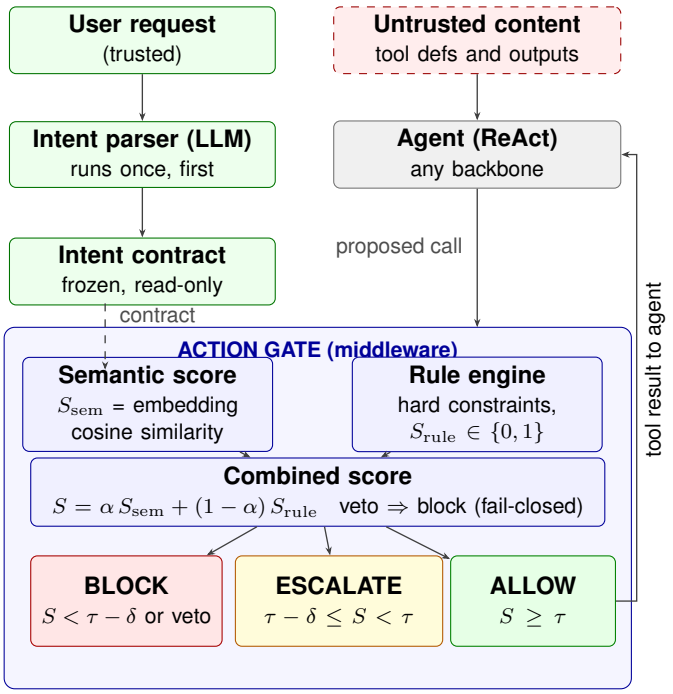


Fig. 1. IntentGate middleware architecture. The intent parser derives a contract once from the trusted request, before any attacker content (tool definitions, tool outputs) is loaded. The gate scores each proposed call with embedding similarity and a hard-rule veto, then decides allow, block, or escalate. Per-action enforcement uses no LLM call.

Algorithm 1 Gate decision for one proposed tool call

Require: contract $C = (A, K, H)$ with embedding e_C; tool call c with embedding e_c; parameters α, τ, δ
Ensure: decision d: allow, block, or escalate
$S_{\text{sem}} \leftarrow \frac{1+\cos(e_C, e_c)}{2}$
$S_{\text{rule}} \leftarrow 1$ if c satisfies A and K, else 0
$S \leftarrow \alpha S_{\text{sem}} + (1-\alpha) S_{\text{rule}}$
if $S_{\text{rule}} = 0$ or $S < \tau - \delta$ **then**
    $d \leftarrow$ block ▷ fail-closed hard veto
else if $S < \tau$ **then**
    $d \leftarrow$ escalate ▷ human-review band
else
    $d \leftarrow$ allow
end if
return d

MCPTox did not result in reduced attack success, because an attack call will look like a normal query. The call itself is legitimate. However, the registration is poisoned. As such, the ToolGate results are statistically indistinguishable from the unprotected baseline results for MCPTox, thus supporting this comparison.

V. Evaluation

A. Benchmarks and cases

InjecAgent [27]: indirect prompt injection into tool-integrated agents, 2,108 cases (data-stealing and direct-harm attacker intents, base and enhanced settings). **MCPTox** [4]: tool-poisoning attacks against real-world MCP servers, 1,348 static-snapshot cases. MCPTox results are scoped to the static snapshot. Live-server testing has been deferred.

B. Metrics

The attack success rate (ASR) is the proportion of attack cases that completed the attacker's objective. For MCPTox, this measure is reported as attack-influenced calls, since the heuristic evaluator identifies payload or sensitive-path evidence. The block rate is the proportion of gated calls blocked or escalated, by stratum where stratified. The valid rate is the proportion of user-tool calls the gate allows, a call-level proxy for utility. Latency reports p50 and p95 per-call gate overhead. Setup cost is the number of hand-authored contracts required.

C. Baselines and ablations

B1 is the unprotected agent. B2 is the ToolGate reimplementation. IntentGate employs $\tau = 0.75$, $\delta = 0.1$, and $\alpha = 0.7$. The full matrix includes three ablations: A1 semantic-only (rule veto disabled), A2 rule-only (embeddings disabled), and A3 raw-request embedding (similarity to the request text instead of the parsed contract). Each condition employed identical prompts, tool blocks, agent model (gpt-4o-mini), embedding model (all-MiniLM-L6-v2 [31]), and seed 42.

D. Statistics

Comparisons used the paired McNemar test with continuity correction [32]. Confidence intervals used a seeded bootstrap with 10k resamples for each benchmark and risk breakdown. The ten risk breakdowns use Bonferroni correction ($\alpha = 0.005$). Post-hoc power is 1.0 at $\alpha = 0.005$ for both benchmarks. Full-matrix results are presented as-is. Pilot results are labeled preliminary.

TABLE II
Headline results (full evaluation matrix). MCPTox reports attack-influenced ASR. Setup cost counts hand-authored contracts.

Condition	InjecAgent ASR	MCPTox ASR	Setup cost	p95 latency
B1 (none)	19.0%	20.7%	n/a	n/a
B2 (ToolGate)	17.2%	20.1%	145 contracts	0.06 ms
Ours (IntentGate)	**0.7%**	**10.7%**	**0**	16–20 ms

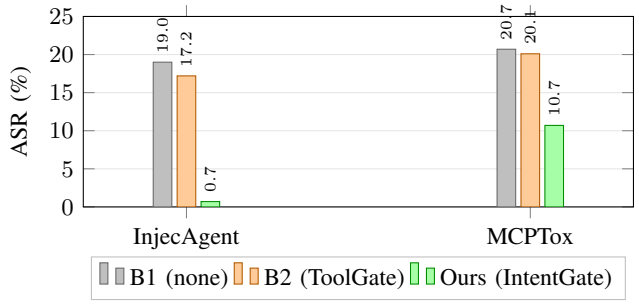


Fig. 2. Attack success rate by condition on the full evaluation matrix (MCPTox reports attack-influenced ASR). IntentGate reaches 0.7% on InjecAgent and 10.7% on MCPTox, while the ToolGate baseline stays close to the unprotected agent.

VI. Results

A. Main results

Table II and Figure 2 summarize the headline result. On InjecAgent, IntentGate reduces ASR from 19.0% to 0.7%. On MCPTox, it reduces attack-influenced calls from 20.7% to 10.7%. On the primary slices (the enhanced data-stealing portion of InjecAgent and the full MCPTox snapshot), McNemar gives $p = 1.3\text{e}{-33}$ and $p = 1.25\text{e}{-07}$. The ToolGate baseline reduces ASR by 1.8 percentage points (17.2% vs. 19.0%) on InjecAgent and is not statistically distinguishable from the unprotected agent on MCPTox. Call-level utility is reported with the benign-task pilot in Section VI-F.

B. Setup cost

IntentGate requires zero hand-authored contracts. ToolGate requires 145 contracts on InjecAgent and 65 on MCPTox (full matrix). Extending the MCPTox set to 583 ToolGate contracts raised weighted coverage to 84.9% without closing the gap. ToolGate's paper does not report this setup-cost metric.

C. Ablations

Semantic scoring alone reaches 0.0% ASR on InjecAgent. The rule-only ablation (A2) is the weakest condition, with 14.7% ASR on InjecAgent and 263 attack-influenced MCPTox calls, close to the unprotected agent on the hard slices. Semantic-only (A1) detects more than the full gate on MCPTox (81 vs. 144 attack-influenced calls) but over-blocks legitimate calls, as the benign-task pilot shows. The raw-request ablation (A3) matches the full gate on InjecAgent

TABLE III
ABLATIONS (FULL EVALUATION MATRIX). A1–A3 REPLACE GATE COMPONENTS. MCPTOX REPORTS ATTACK-INFLUENCED CALLS (OUT OF 1,348).

Condition	InjecAgent ASR	MCPTox calls	Notes
A1 (semantic-only)	0.0%	81	Detects more. Over-blocks legitimate calls
A2 (rule-only)	14.7%	263	Closest to unprotected on hard slices
A3 (raw-request)	0.7%	116	Equal to ours on InjecAgent. Fewer MCPTox calls
Ours (full)	**0.7%**	**144**	Contract adds utility and auditability

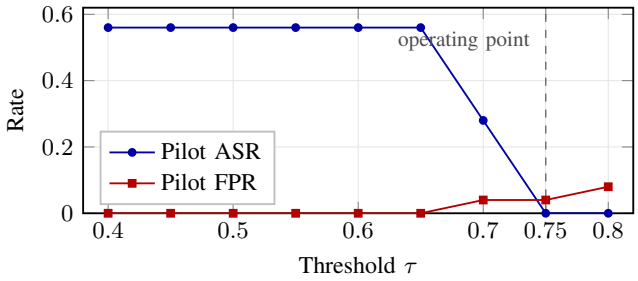


Fig. 3. Threshold sweep on the 200-case pilot ($\delta = 0.1$): attack success rate (ASR) and false-positive rate (FPR) versus τ. The operating point $\tau = 0.75$ sits at the knee: ASR reaches zero while FPR stays at 4%.

(0.7% both) and catches slightly more attacks on MCPTox (116 vs. 144 attack-influenced calls). The contract's value is therefore utility and auditability. Headline ASR is similar. The false-escalate rate is 14.8% over those 583 escalated calls.

D. Pareto and threshold

Figure 3 shows the threshold sweep on the pilot data: attack success falls to zero at the operating point $\tau = 0.75$ while the false-positive rate stays at 4%. The operating point is fixed for the full matrix, and threshold transfer across encoders and splits is not claimed.

E. Error taxonomy

Thirteen InjecAgent failures remain. All are read-only exfiltration calls with $S \in [0.75, 0.79]$, just above τ. All 13 were already allowed, and rewording them to sound like the user's request raised S by 0.0112 on average without moving any additional case across τ. These prose variants therefore do not change the operating point. The remaining MCPTox failures cluster in poisoned-registration cases whose calls look like legitimate queries, so no request-level rule fires.

F. Validation results (preliminary)

A second backbone, qwen3.5:4b, replicates the ASR reduction on InjecAgent (27.1% $\rightarrow$ 2.1%, $n = 100$ per condition). gpt-6-luna is excluded because its prose-only output never participates in the ReAct-style loop. On 2,013 gated calls, the block rate is 12.8% for specific contracts, 9.6% for vague contracts, and 12.4% overall.

TABLE IV
UTILITY AND VALIDATION PILOTS (PRELIMINARY): BENIGN-TASK UTILITY, SECOND BACKBONE, AND PROMPT-LEVEL BASELINES.

Pilot	Result
Benign utility	user-tool allowed: none 16.0%, ours 12.0%, semantic-only 8.0%. Clean-call block: none 0.0%, ours 25.0%, semantic-only 42.9%
Second backbone	qwen3.5:4b: ASR 27.1% $\rightarrow$ 2.1%
Prompt baselines	delimiters 4.3%, sandwich 4.3%, no defense 6.8%, instructional prevention 8.9% (50 cases each)

The benign-task pilot (50 clean cases) compares the unprotected agent, the full gate, and the semantic-only ablation. User-tool allowed rates are 16.0%, 12.0%, and 8.0%. Clean-call block rates are 0.0%, 25.0%, and 42.9%. Semantic-only over-blocks legitimate calls, so the contract preserves more of them in this comparison.

The prompt-baseline pilot tests 50 attack cases per condition. ASR is 6.8% with no defense prompt, 4.3% with delimiters, 4.3% with sandwiching, and 8.9% with instructional prevention. DeepSeek V4.1 Flash refused the static injections (0/100 in both conditions), so its delta is undefined. This is a model-level refusal, not a gate effect. Phrasings that change how the injection is delivered stayed at or below 2.2% gated ASR. All pilot numbers are preliminary.

VII. LIMITATIONS

Static attacks were studied. There was no assertion to the effect that adaptive attackers would be able to perform better than the static attacker. An offline probe can create a copy of the user's request into free-text tool arguments and cause S to exceed τ. A single backbone (gpt-4o-mini) is used for all elements of the full matrix. qwen3.5:4b replicates the delta found in InjecAgent.

Contract coverage for MCPTox tools that are outside the subset evaluated in ToolGate remains low by design. Valid rate refers to the percent of the user's requests to use tools allowed by the system and not overall success of completing tasks. The MCPTox attack-influenced metric is obtained through a heuristic evaluator (payload and sensitive path evidence) and therefore is less detailed than an objective evaluator.

VIII. CONCLUSION

The results show that an action gate can derive its policy automatically from the user's request and enforce it with embedding similarity plus a deterministic veto at the tool-call chokepoint. The gate needs no hand-authored per-tool contracts, adds 16–20 ms p95 latency, and reduces attack success on an injection and a tool-poisoning benchmark. The reproducible artifact is released after acceptance. Future work covers adaptive-attacker hardening, live-server MCPTox evaluation, and multi-turn drift.

ACKNOWLEDGMENT

The authors thank their supervisor for supervision, guidance, and continuous support throughout this work. Large

language models were used only for language polishing, such as grammar and spelling correction.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[3] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," 2022, NeurIPS 2022 ML Safety Workshop. [Online]. Available: https://arxiv.org/abs/2211.09527

[4] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X.-Y. Li, "MCPTox: A benchmark for tool poisoning on real-world MCP servers," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026, pp. 35 811–35 819. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/40895

[5] B. Radosevich and J. Halloran, "MCP safety audit: LLMs with the Model Context Protocol allow major security exploits," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2504.03767

[6] OWASP Foundation, "OWASP top 10 for large language model applications," 2025, available: https://genai.owasp.org/llm-top-10/.

[7] M. A. Ferrag, N. Tihanyi, D. Hamouda, L. Maglaras, A. Lakas, and M. Debbah, "From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows," *ICT Express*, vol. 12, pp. 353–383, 2026.

[8] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI agents with privilege control," 2025, preprint, version 3, revised May 14, 2026. [Online]. Available: https://arxiv.org/abs/2504.11703v3

[9] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design (CaMeL)," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2503.18813

[10] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium (USENIX Security)*, 2025, pp. 2383–2400. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25

[11] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "SecAlign: Defending against prompt injection with preference optimization," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025, pp. 2833–2847.

[12] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," 2024, preprint. [Online]. Available: https://arxiv.org/abs/2404.13208

[13] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," 2024, preprint. [Online]. Available: https://arxiv.org/abs/2403.14720

[14] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2023, pp. 431–445.

[15] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, "Prompt injection attack to tool selection in LLM agents," 2025, preprint, version 3. [Online]. Available: https://arxiv.org/abs/2504.19793v3

[16] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023, preprint. [Online]. Available: https://arxiv.org/abs/2307.15043

[17] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[18] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025, pp. 1809–1820.

[19] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, "MELON: Provable defense against indirect prompt injection attacks in AI agents," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. [Online]. Available: https://arxiv.org/abs/2502.05174

[20] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, 2025, pp. 29 680–29 697.

[21] Y. Liu, X. Peng, J. Cao, X. Wang, S. Deng, J. Chen, J. Yin, and X. Zhang, "ToolGate: Contract-grounded and verified tool execution for LLMs," 2026, preprint. [Online]. Available: https://arxiv.org/abs/2601.04688

[22] W. Zhao, Z. Li, P. Zhang, and J. Sun, "ClawGuard: A runtime security framework for tool-augmented LLM agents against indirect prompt injection," 2026, preprint. [Online]. Available: https://arxiv.org/abs/2604.11790

[23] J. Kim, W. Choi, and B. Lee, "Prompt flow integrity to prevent privilege escalation in LLM agents," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2503.15547

[24] W. Hua, X. Yang, M. Jin, Z. Li, W. Cheng, R. Tang, and Y. Zhang, "TrustAgent: Towards safe and trustworthy LLM-based agents," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.585/

[25] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Paverd, F. Tramèr, and V. Volhejn, "Design patterns for securing LLM agents against prompt injections," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2506.08837

[26] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. [Online]. Available: https://arxiv.org/abs/2406.13352

[27] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. [Online]. Available: https://aclanthology.org/2024.findings-acl.624/

[28] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, Q. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2410.02644

[29] Anthropic, "Introducing the Model Context Protocol," 2024, available: https://www.anthropic.com/news/model-context-protocol.

[30] Q. Zhan, R. Fang, H. S. Panchal, and D. Kang, "Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 7101–7117. [Online]. Available: https://aclanthology.org/2025.findings-naacl.395/

[31] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3980–3990.

[32] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.